

Test Task: Meeting Room Booking System with Concurrency Control

Objective: Build a booking system for a limited set of resources (meeting rooms) where multiple users may try to book the same time slot at the same time. The system must guarantee that a slot is never double-booked, and must reflect booking status to all viewers in real time.

Notes: For the backend, use ASP.NET Core. Frontend framework is your choice. Development should be carried out with active use of Claude (Claude Code) — the repository must reflect this (see item 9).

Steps:

1. Register an account on Azure Portal

- Create an account if you don't already have one.

2. Set up Azure resources

- Azure Web App for backend and frontend.
- Azure SignalR Service.
- Azure SQL Database.

3. Authorization and roles

- Implement user authentication with a role-based model.
- Regular user: can view resources and their schedules, and book available slots.
- Admin: can additionally create, edit, and remove resources, and view all bookings across users.

4. Resources and scheduling

- A resource (e.g. a meeting room) has a fixed set of bookable time slots.
- Users can view, for any resource, which slots are free and which are booked.

5. Booking with concurrency control

- **Requirements:**
 - When a user requests to book a slot, the system must guarantee that if two (or more) requests for the same slot arrive at effectively the same time, exactly one succeeds and the rest receive a clear conflict response — never a silent overwrite and never a server error.
 - The chosen approach to concurrency (e.g. transactional locking, optimistic concurrency with versioning, or another explicit mechanism) is up to the candidate, but must be a deliberate, explained design decision — not left to incidental behavior of the ORM or database defaults.
 - A naive “check if free, then insert booking” implemented as two separate, unprotected steps does not satisfy this requirement.

6. Automated concurrency test

- **Requirements:**
 - Include an automated test (or small test script) that fires multiple simultaneous booking requests at the same slot and asserts that exactly one booking is created.
 - This test must be part of the submitted repository and runnable by the reviewer.

7. Real-time updates

- **Requirements:**
 - Use Azure SignalR Service so that when a slot's booking status changes, everyone currently viewing that resource's schedule sees the update immediately, without refreshing the page.

8. Deployment

- Deploy the backend and frontend applications to Azure.

9. Repository and development process requirements

- Commit history must be atomic: each commit includes a short description of what changed and why.
- The repository must contain a Claude Code configuration that reflects the use of Claude in the development process (CLAUDE.md; optionally, custom skills that speed up the development itself).
- Code must be well-documented and suitable for review.

10. Submitting the result

- Send a link to the deployed Azure Web Application.
- Send a link to the GitHub repository with the source code.
- Make sure the source code is well-documented and accessible for review.